Particle Filter Implementation

Caleb Sparks

December 2022

1 Introduction

The RC subteam wants to implement an on board localization system. To do this, a "poor man's lidar" system will be created out of four time of flight distance sensors placed on each side of the car. The sensors will report distance to the track walls in each of the 4 directions, as shown in Figure 1. "Particle Filters: A Hands-On Tutorial" ("the paper") provides an overview of particle filters a practical guide for implementing them and is the paper I followed when creating this one [1].

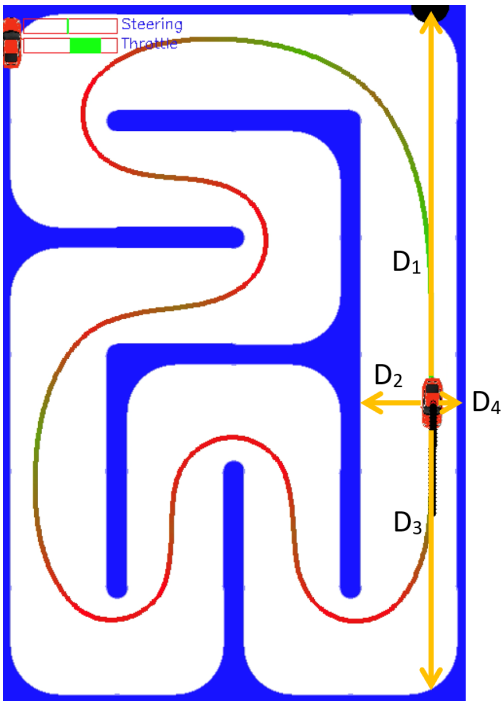

Figure 1: Poor Man's Lidar Setup

2 Generic Algorithm

The particle filter I created for the RC car is based on Algorithm 3 in the paper, shown in Figure 2. The (13) is a reference to the paper's Equation 13, shown here:

$$w_k^i \propto w_{k-1}^i \frac{p(z_k|x_k^i)p(x_k^i|x_{k-1}^i)}{q(x_k^i|x_{k-1}^i, z_k)},$$

where i the particle, k represents the time step, x represents the state, z represents the measurement, and w represents the weight. As indicated in the paper, the importance function $q(x_k^i|x_{k-1}^i, z_k)$ is most commonly chose to be $p(x_k^i|x_{k-1}^i)$, meaning Equation 13 simplifies to Equation 16,

$$w_k^i \propto w_{k-1}^i p(z_k|x_k^i),$$

which, in the context of the RC car, says that the current particle weight is proportional to the weight at the previous time step multiplied by the probability that the distance measurements reported by the sensors would be observed given the particle's position.

Algorithm 3: Particle filter with resampling threshold

Input: $\{x_{k-1}^i, w_{k-1}^i\}_{i=1}^{N_s}, z_k$
Output: $\{x_k^i, w_k^i\}_{i=1}^{N_s}$
$w_{sum} = 0$
for $i = 1, \ldots, N_s$ **do**
  /* Step 1: propagate particle */
  draw sample $x_k^i \sim q(x_k^i \mid x_{k-1}^i, z_k)$
  /* Step 2: update weight */
  assign weight w_k^i using (13)
  /* Step 3: cumulative weight */
  $w_{sum} = w_{sum} + w_k^i$
end
/* Normalize weights */
for $i = 1, \ldots, N_s$ **do**
  $w_k^i = w_k^i / w_{sum}$
end
if $\hat{N}_{eff} < N_{threshold}$ **then**
  /* Resample if needed */
  Resample N_s particles with replacement
  /* Reset weights */
  for $i = 1, \ldots, N_s$ **do**
    $w_k^i = 1/N_s$
  end
end

Figure 2: Algorithm 3: Particle Filter with Resampling Threshold

Algorithm 3 addresses two potential challenges associated with particle filters: particle degeneracy and sample impoverishment. Both issues are explained in the paper. Resampling addresses particle degeneracy and reducing resampling frequency by using a threshold addresses sample impoverishment. Additionally, resampling is computationally expensive, so doing so less frequently can increase the code's frequency. For these reasons, I selected this algorithm.

3 Implementation

My particle filter is a particular implantation of the more general Algorithm 3. The steps below detail the specifics of my implementation. I have provided descriptions of the steps, in most cases followed by an

equation showing what the code does.

- Initialize (particle_filter class __init__ method)

  1. Set translation process standard error, q_{trans}
  2. Set rotation process standard error, q_{rot}
  3. Set the process noise covariance matrix

     $$\mathbf{Q} = \begin{bmatrix} q_{trans}^2 & 0 & 0 \\ 0 & q_{trans}^2 & 0 \\ 0 & 0 & q_{rot}^2 \end{bmatrix}$$

  4. Set sensor standard error, r. (Assume all sensors are equivalent)
  5. Set the sensor noise covariance matrix

     $$\mathbf{R} = \begin{bmatrix} r^2 & 0 & 0 & 0 \\ 0 & r^2 & 0 & 0 \\ 0 & 0 & r^2 & 0 \\ 0 & 0 & 0 & r^2 \end{bmatrix}$$

  6. Set threshold for effective number of particles, N_{eff}

     $N_{eff} = \frac{1}{4}N_s$
  7. Initialize particles, P, from either (by taking N_s samples):
     (a) A 2D uniform distribution over the whole track (no initial information), or
     (b) A 2D normal distribution at the car's starting position.

  Note: A covariance matrix for error in placing the car at the starting position is needed for the normal distribution. It takes the same form as $\mathbf{Q}$.

     $$\mathbf{P}_0 = \begin{bmatrix} x_0^1 & y_0^1 & \theta_0^1 \\ \vdots & \vdots & \vdots \\ x_0^{N_s} & y_0^{N_s} & \theta_0^{N_s} \end{bmatrix}$$

  8. Set weights to $\frac{1}{N_s}$

     $$\mathbf{W_0} = \begin{bmatrix} w_0^1 = \frac{1}{N_s} \\ \vdots \\ w_0^{N_s} = \frac{1}{N_s} \end{bmatrix}$$

- Propagate all particles through dynamics model and add process noise

  1. Get control inputs, $\mathbf{u_{k-1}}$
  2. Use dynamics to calculate the new state for the actual car

     $\mathbf{x_k} \leftarrow \text{DynamicsModel}(\mathbf{x_{k-1}}, \mathbf{u_{k-1}})$

  Note: In the real implementation, the car's state would not be available, but the choice to advance this state and use its as the "master state" was arbitrary. Any position and heading could be used as input; what matters is the change between the states.

  3. Calculate the change between the states, $\Delta\mathbf{x_k}$
  4. Apply a transform to calculate the change between states for each particle, $\Delta\mathbf{X_k}$ (shape is $3N_s \times 1$)

     $$\Delta\mathbf{X_k} = T\Delta\mathbf{x}, T = \begin{bmatrix} \cos(\theta_{k-1}^1 - \theta_{k-1}) & -\sin(\theta_{k-1}^1 - \theta_{k-1}) & 0 \\ \sin(\theta_{k-1}^1 - \theta_{k-1}) & \cos(\theta_{k-1}^1 - \theta_{k-1}) & 0 \\ 0 & 0 & 1 \\ \vdots & \vdots & \vdots \\ \cos(\theta_{k-1}^{N_s} - \theta_{k-1}) & -\sin(\theta_{k-1}^{N_s} - \theta_{k-1}) & 0 \\ \sin(\theta_{k-1}^{N_s} - \theta_{k-1}) & \cos(\theta_{k-1}^{N_s} - \theta_{k-1}) & 0 \\ 0 & 0 & 1 \end{bmatrix}_{3N_s \times 3}, \Delta\mathbf{x_k} = \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta\theta \end{bmatrix}$$

Note: All can be advanced at once because all are assumed to have the same velocity and angular velocity; only the direction of translation must be adjusted according to each particle's heading relative to the "master" heading, θ_{k-1}.

5. Reshape $\Delta\mathbf{X_k} to N_s \times 3\Delta\mathbf{X_k} = \begin{bmatrix} \Delta x_k^1 & \Delta y_k^1 & \Delta\theta_k^1 \\ \vdots & \vdots & \vdots \\ \Delta x_k^{N_s} & \Delta y_k^{N_s} & \Delta\theta_k^{N_s} \end{bmatrix}$
6. Add each change to the corresponding particle

   $\mathbf{P_k} = \mathbf{P_{k-1}} + \Delta\mathbf{X_k}$
7. Add process noise

   $\mathbf{P_k} \leftarrow \mathbf{P_k} + \mathbf{q}, \mathbf{q} = \begin{bmatrix} [q_x^1 & q_y^1 & q_\theta^1] \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \\ \vdots & \vdots & \vdots \\ [q_x^{N_s} & q_y^{N_s} & q_\theta^{N_s}] \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \end{bmatrix}$

Note: This is the end of the preUpdate method of the class ParticleFilter.

- Calculate weights

1. Get sensor measurements (simulated for now)

   $\mathbf{M}_{1\times 4} \leftarrow SimulateSensorsOnlyNums(\mathbf{x_k})$
2. Add sensor noise

   $\mathbf{M} \leftarrow \mathbf{M} + \mathbf{r}, \mathbf{r} = \begin{bmatrix} r_1 & r_2 & r_3 & r_4 \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$
3. Simulate measurements expected by particles

   $\mathbf{M}_{expected} = [\mathbf{0}]_{N_s \times 4}$

   for $i = 1, ..., N_s$

   $\mathbf{M}_{expected}[i] = SimulateSensorsOnlyNums(\mathbf{P_k}[i])$

Note: This is currently done by inching forward by fractions of a centimeter until a wall is reached, which is very slow. Worse, it must be done as many times as there are particles. This process should be improved.

4. Calculate likelihood of each particle observing the measurements from the sensors given the location of the particle (i.e., given the expected measurement from the particle's location)

   $\mathbf{L}_{N_s\times 4} = n(\mathbf{M}_{expected} - \begin{bmatrix} \mathbf{M} \\ \vdots \\ \mathbf{M} \end{bmatrix}_{N_s \times 4})$, where n is the probability density function of a normal distribution with mean $\mu = 0$ and standard deviation $\sigma = r$.

   $\mathbf{L}_{combined} = \prod_{j=1}^{4} \mathbf{L}[:][j]$ (element-wise multiplication of columns)
5. Calculate posterior weights using Equation 16

   $\mathbf{W_k} = \mathbf{W_{k-1}} \cdot \mathbf{L}_{combined}$
6. Sum the weights and normalize

   $\mathbf{W_k} \leftarrow \frac{1}{\sum_{i=1}^{N_s} \mathbf{W_k}[i]} \mathbf{W_k}$

- Calculate particle filter estimate as the weighted average of all particle states

  $\mathbf{P}_{\mathbf{k},best} = \begin{bmatrix} \mathbf{P_k}_x \cdot \mathbf{W_k} & \mathbf{P_k}_y \cdot \mathbf{W_k} & \mathbf{P_k}_\theta \cdot \mathbf{W_k} \end{bmatrix}$, where, for example, $\mathbf{P_k}_x$ is the first column of $\mathbf{P_k}$, which holds the particle's x-axis locations.

Note: If particles are widely spread, like for a short time after a uniform initialization, the best estimate will likely be in an area far from clusters of particles. However, this greatly reduces the "jitter" associated with using the ever-changing highest weighted particle as the estimated state when the filter has converged, which is the exceedingly more common situation.

- Resample if necessary
  1. Calculate the effective number of particles: $N_{eff} = \frac{1}{\sum_{i=1}^{N_s}(w_k^i)^2}$, where N_s is the number of particles
  2. If N_{eff} is less than the threshold, resample: set all weights $w_i = \frac{1}{N_s}$

4 Additional Comments and Next Steps

There are several important things not yet discussed. First, a few rapid-fire housekeeping items and points of clarification.

- In the code, **P** and **W** are combined as [**P W**].
- In the code, the differences between each particle's heading and the master heading are stored in a $N_s \times 1$ array. This array is input to the proper cos and sin functions and distributed to every third row of the T transform matrix, which is divided into 3×3 matrices corresponding to each particle.
- The number of particles used is set in the main run.py file using 'numberOfParticles' in the 'params' dict variable.
- The particle filter is an "Extension" in its own right, as the ParticleFilter class. This class serves primarily to wrap the actual particle filter (the class particle_filter) in a class that extends Extension so that things play nice in the overall pipeline.
- Although the ParticleFilter class improves the situation, there is still a snag with visualization. To visualize the best estimate of the particle filter with a car, I add an additional car to the simulation. I did this to have easy access to all the same methods that allow me to draw it, but a better method should probably be used. To prevent it from being rendered normally, I have prevented all cars other than the first from being visualized normally. I then use the second car in methods I wrote myself for visualizing the particle filter, which could also probably be improved.
- I have comments in the code about needing to address if the car is facing the wrong way. I believe this should not actually be an issue. (I was concerned about the left sensor being switched with the right, and forward with backward, but simulating measurements based on heading should solve this issue.)
- The sensors are not necessarily (and rather probably not) set up in the order indicated in Figure 1.

Second, note that this algorithm assumes that velocity and angular velocity are known. In the real implementation, the velocity would have to be determined. This is the first of the next steps. This could be done by including these velocities in the state being estimated by the particle filter. A choice would then have to be made to either use the best estimate for velocity for all particles when advancing them through the dynamics model or to propagate each with their own velocity. The former would allow all particles to be updated at once through a transformation, saving computation, but loses some particle diversity by using the same velocity. The latter is closer to how particle filters are typical configured and could better resist sample impoverishment, but would require more computation. In either case, it is likely that more particles will be needed because a distribution with three additional degrees of freedom must be represented.

Third, the particle filter is terribly slow. It can't finish a lap in under a minute, much less a few seconds. There are two things (that I know of) to improve. First, as mentioned before, simulation of particles' measurements must be faster. I believe this could be done using a simpler version of rasterization, which is used to render video games. It provides an efficient way to determine how far away an object is in 3D space and therefore which objects to display on your screen. For the RC car's localization purposes, we are only concerned with the walls of the track, which are normal to the sensors' emitted laser, which may mean only two dimensions are required. See this website, Scratchapixel, for information on the rasterization algorithm [2]. I anticipate this would require a fair bit of work because the track walls would have to be rendered and stored as shapes that can be used by the rasterization algorithm.

However, it is also likely that the step sizes taken by the current algorithm could be optimized in real time, which may prove to be easier. Numerical root finding methods, particularly the Bisection method,

come to mind as inspiration. Second, the particles and their movement are (for the most part) independent of each other. This means that parallel processing can be used to speed computation. This applies to both multiplication of matrices and (especially) the sensor simulation problem.

The process and sensor noise matrices should be determined experimentally. What is the true ability of the dynamic model? I used a ballpark guess. How well do the sensors perform? Moreover, how well do they perform on a moving, vibrating vehicle while looking for small, short walls? (Do the wall heights need to be increased?) Again, I used my best guess, but accurate values will be needed.

Finally, the sensors will not work exactly as the particle filter assumes. The sensors will be continuously running, updating the four directional measurements. The measurements, therefore, will not necessarily be from exactly the same moment for which the dynamics model calculated a position, and the four measurements will be taken at different times. This comes as a result of using interrupts, but the alternative is halting the code to wait for sensor measurements (polling), which is probably not acceptable. A method to determine how old the measurement is may be required. If it is, how will a newer measurement made more recently be compared an older one? Or with the dynamics model's process noise? My intuition is that the answer comes in the form of a sensor noise model that is dependent on how long ago the measurement was taken (noise could be increased with time, but should it be increased linearly, exponentially, or none of the above?). The methods that yield the best performance will likely come from a combination processing measurements on the Arduino and adapting the particle filter to incorporate more complicated sensors.

The particle filter (and Kalman filter) are some pretty interesting projects for the RC car team going forward. As I've outlined, there are many open ended questions to be answered and great opportunity to learn about state estimation, Bayesian logic, parallel processing, graphics, and low-level coding in the process. This would be a great project to continue pursuing because future VIP students should be able to get a lot out of it.

References

[1] Jos Elfring, Elena Torta, and René van de Molengraft. "Particle Filters: A Hands-On Tutorial". In: *Sensors* 21.2 (2021). ISSN: 1424-8220. DOI: 10.3390/s21020438. URL: https://www.mdpi.com/1424-8220/21/2/438.

[2] "Rasterization: a Practical Implementation". URL: https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/overview-rasterization-algorithm.